Enfoques del Desarrollo de Software aplicados al Sistema de Gestión de Matrículas Académicas de la FET

[Nombres completos de los integrantes del equipo]
Fundación Escuela Tecnológica de Neiva
[Nombre del programa y del curso]
[Nombre del docente]
11 de noviembre de 2026

Enfoques del Desarrollo de Software aplicados al Sistema de Gestión de Matrículas Académicas de la FET

La Fundación Escuela Tecnológica de Neiva (FET) todavía maneja varios trámites de matrícula a mano o en hojas de cálculo, y eso trae errores, demoras y problemas para saber cuántos cupos quedan. En este trabajo planteamos, como si fuéramos el equipo de ingeniería de software, un primer análisis y diseño de un sistema de matrículas usando los enfoques estructurado, orientado a objetos y ágil (Sommerville, 2011; Pressman, 2010).

Parte A. Enfoque estructurado

Procesos principales del sistema

El enfoque estructurado parte el problema en procesos que reciben datos y entregan resultados (Sommerville, 2011). En la Tabla 1 listamos los cinco procesos que nos parecen centrales según lo que pide la FET.

Tabla 1

Procesos funcionales principales del sistema de matrículas

N.º	Proceso funcional	Descripción
1	Gestión de estudiantes	Registrar estudiantes y consultar o actualizar sus datos personales y académicos.
2	Consulta de oferta académica	Consultar asignaturas, horarios y cupos disponibles por período.
3	Validación de matrícula	Verificar prerrequisitos y detectar conflictos de horario antes de inscribir.
4	Matrícula y cancelación	Inscribir o cancelar asignaturas y actualizar el cupo de cada grupo.
5	Comprobantes y reportes	Generar el comprobante de matrícula y los reportes académicos.

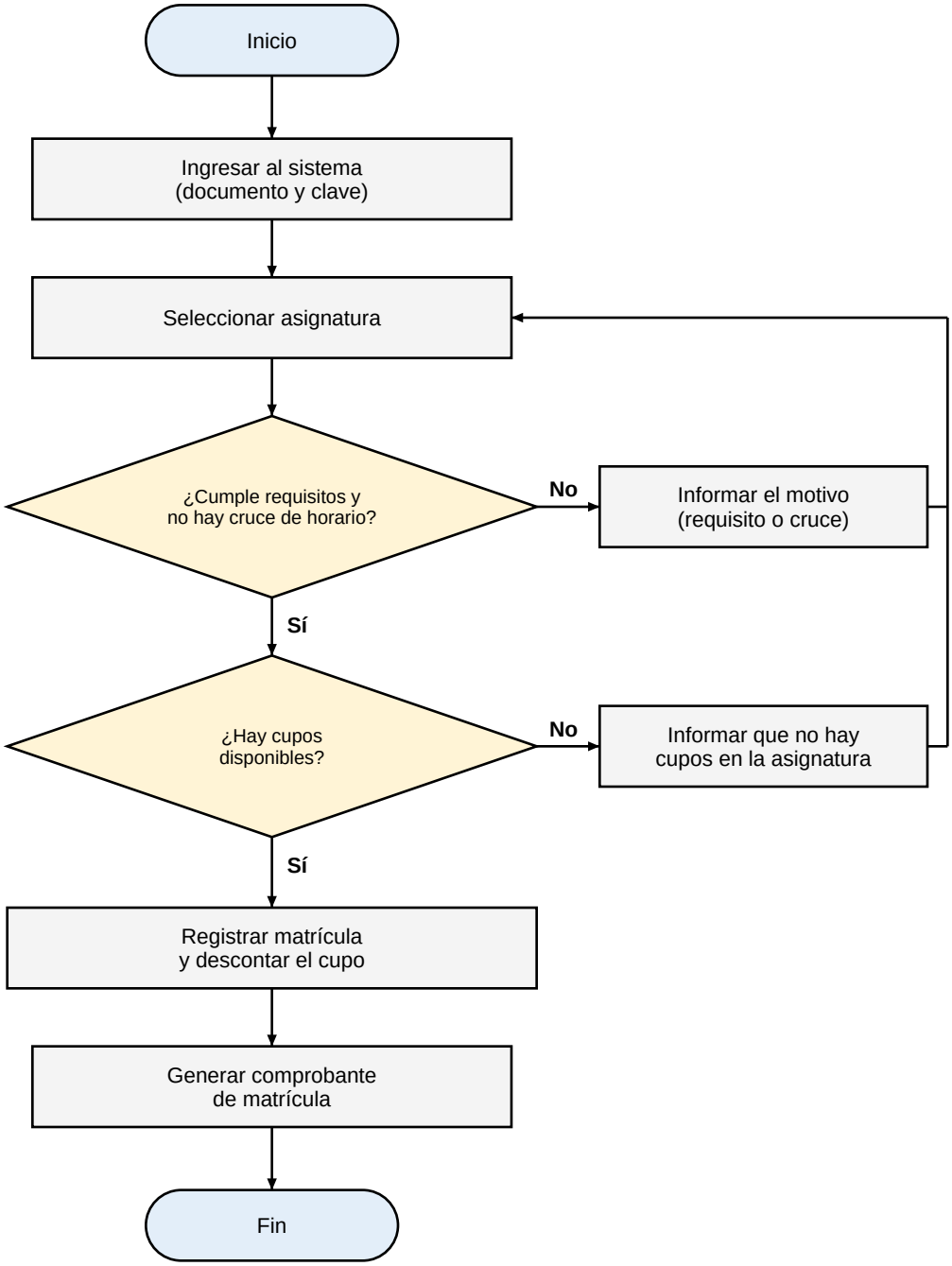
Nota. Elaboración propia a partir de la situación problema planteada.

Diagrama de flujo

La Figura 1 muestra el camino del estudiante desde que ingresa hasta que recibe el comprobante. Aparte de la pregunta por los cupos, agregamos la validación de requisitos y horarios porque hace parte del proceso 3 y evita matricular algo que luego toque anular.

Figura 1

Diagrama de flujo del proceso de matrícula



Nota. Las ramas «No» regresan al paso de selección para permitir elegir otra asignatura. Elaboración propia.

Parte B. Enfoque orientado a objetos

La orientación a objetos representa el sistema como objetos que guardan sus propios datos y saben hacer ciertas operaciones, y esos objetos se agrupan en clases (Booch et al., 2006). En la Tabla 2 están los atributos y métodos de las tres clases sugeridas.

Tabla 2

Atributos y métodos de las clases del sistema

Clase	Cuatro atributos	Tres métodos
Estudiante	idEstudiante nombres programa correo	registrarEstudiante() consultarDatos() actualizarDatos()
Asignatura	codigo nombre horario cupoDisponible	consultarHorario() verificarCupo() verificarRequisitos()
Matrícula	idMatricula fecha estado periodoAcademico	matricular() cancelar() generarComprobante()

Nota. Elaboración propia.

Preguntas de análisis

¿Qué diferencia hay entre clase y objeto? La clase es el molde: define qué datos y qué operaciones tiene cualquier estudiante, por ejemplo. El objeto es un caso concreto de ese molde, como un estudiante en particular con su código y su correo.

¿Por qué importa el encapsulamiento? Porque los datos quedan dentro de la clase y solo se cambian por medio de sus métodos. El cupo de una asignatura, por ejemplo, no debería poder modificarse desde cualquier parte del programa sino solo cuando alguien se matricula o cancela. Así se evitan errores y es más fácil cambiar el código sin dañar lo demás.

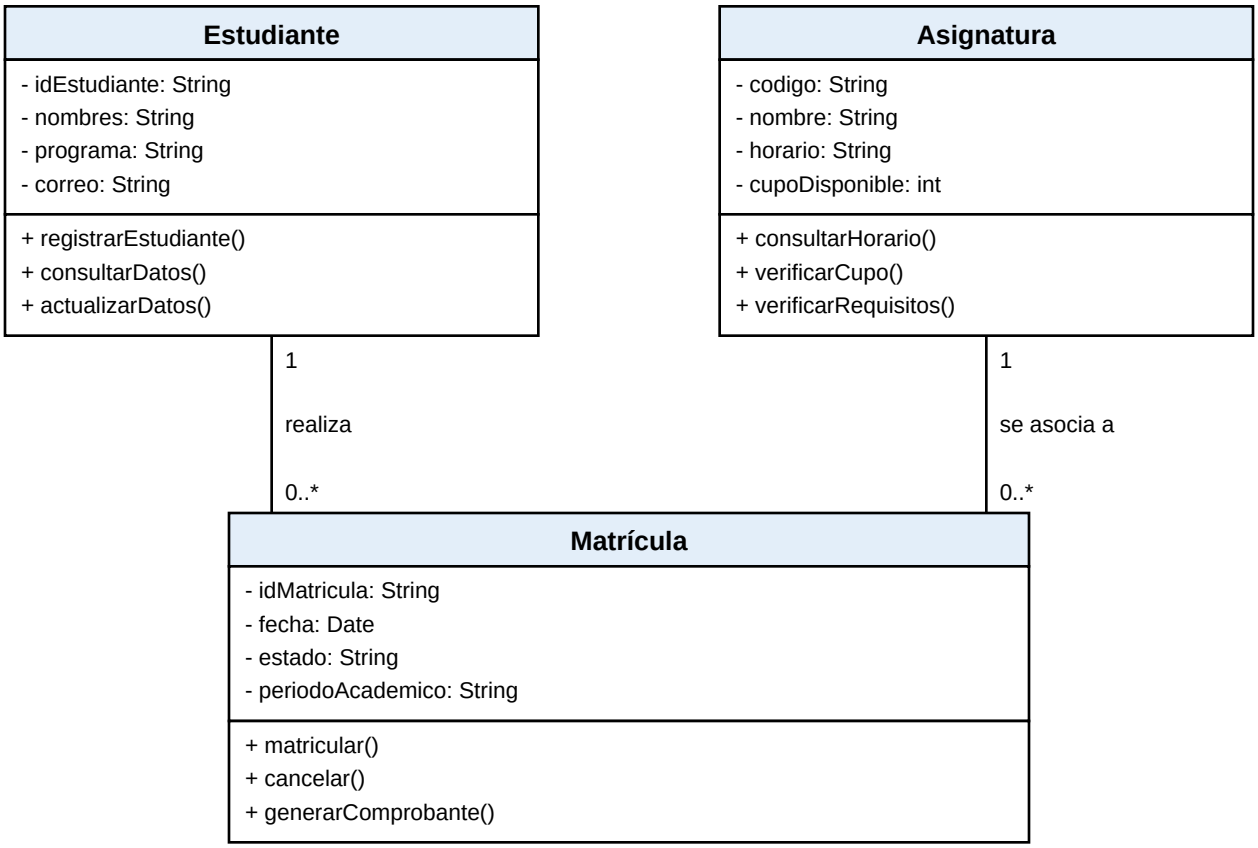
¿Cómo se relacionan Estudiante, Asignatura y Matrícula? Un estudiante puede tener varias matrículas, pero cada matrícula es de un solo estudiante. Con la asignatura pasa igual: aparece en muchas matrículas y cada matrícula apunta a una sola asignatura. Por eso Matrícula queda en el medio y resuelve la relación de muchos a muchos entre Estudiante y Asignatura.

Diagrama de clases UML

En la Figura 2 aparece el diagrama de clases con lo anterior. El signo «-» marca los atributos privados y el «+» los métodos públicos, que es justo la idea del encapsulamiento.

Figura 2

Diagrama de clases UML del sistema de matrículas



Nota. Las multiplicidades indican cuántos objetos participan en cada relación. Elaboración propia.

Parte C. Enfoque ágil

El enfoque ágil entrega el sistema por partes, en ciclos cortos, y va ajustando el rumbo con lo que opinan los usuarios (Beck et al., 2001). En la Tabla 3 proponemos cuatro Sprints de dos semanas, cada uno con algo que se pueda probar en la revisión.

Tabla 3

Plan de cuatro Sprints de dos semanas

Sprint	Funcionalidad priorizada	Resultado / incremento verificable
1	Registro y consulta de estudiantes, con ingreso al sistema.	Un estudiante de prueba se registra, inicia sesión y consulta sus datos; las pruebas del módulo se ejecutan sin errores.
2	Consulta de asignaturas, horarios y cupos disponibles.	Catálogo consultable por programa y período que muestra horarios y el cupo actualizado de cada grupo.
3	Matrícula y cancelación de asignaturas con verificación de requisitos y cupos.	Un estudiante matricula y cancela asignaturas, el sistema rechaza los casos sin cupo o sin requisitos y el cupo se actualiza correctamente.
4	Comprobantes y reportes académicos.	Comprobante en PDF y reporte de matriculados y cupos por asignatura, validados con la oficina de registro en pruebas de aceptación.

Nota. Elaboración propia.

Respuesta al cambio solicitado en el Sprint 2

Cómo analizaríamos la solicitud. La tomaríamos como una nueva historia de usuario. Hablaríamos con los estudiantes para entender qué consideran un cruce y cómo quieren que les avise el sistema, definiríamos cuándo se da por terminada y los desarrolladores estimarían cuánto trabajo implica. También veríamos que se parece a la validación que ya está planeada para el Sprint 3.

Quién prioriza. Lo decide el Product Owner, que responde por el valor del producto y ordena el Product Backlog; para eso escucharía a los estudiantes y a la oficina de registro (Schwaber y Sutherland, 2020).

Qué pasa con el trabajo en curso. No pararíamos el Sprint 2: se termina el catálogo de asignaturas y cupos, que es su objetivo. La historia se anota en el Product Backlog y, si el Product Owner la ve urgente, se programa en el Sprint 3 junto con la verificación de requisitos. Solo si el cambio dejara sin sentido el objetivo del Sprint, el Product Owner podría cancelarlo; en ese caso se guarda lo que ya esté terminado y se planea de nuevo lo que falta.

Parte D. Comparación y reflexión

En la Tabla 4 comparamos los tres enfoques.

Tabla 4

Matriz comparativa de los enfoques de desarrollo

Aspecto	Estructurado	Orientado a objetos	Ágil
Característica principal	Divide el sistema en procesos y flujos de datos con una secuencia clara.	Modela el sistema con objetos que unen datos y comportamiento.	Entrega por iteraciones cortas con cambio y colaboración constantes.
Ventaja	Procesos fáciles de entender, documentar y validar con los usuarios.	Código reutilizable, modular y de fácil mantenimiento.	Entregas tempranas y adaptación rápida a nuevos requisitos.
Desventaja	Poco flexible ante cambios y con datos y procesos separados.	Exige más abstracción y diseño inicial cuidadoso.	Requiere participación continua de los usuarios y poca documentación formal.
Aplicación al caso	Modelar el flujo de matrícula y los cinco procesos de la FET.	Diseñar las clases Estudiante, Asignatura y Matrícula.	Construir el sistema en cuatro Sprints y atender solicitudes como la detección de cruces.

Nota. Elaboración propia a partir de Sommerville (2011), Pressman (2010) y Beck et al. (2001).

Conclusión argumentada

Para la FET usaríamos los tres enfoques juntos, cada uno en lo que mejor hace. El estructurado sirve al comienzo para entender y dibujar los procesos reales, como revisar cupos o entregar el comprobante, y confirmarlos con la oficina de registro. El orientado a objetos ayuda a diseñar un sistema ordenado, con clases como Estudiante, Asignatura y Matrícula que cuidan sus propios datos y son más fáciles de mantener. Y el ágil permite construirlo por Sprints, mostrar avances reales y recibir comentarios de estudiantes y funcionarios, de modo que un cambio como detectar cruces de horario se pueda incluir sin rehacer todo. Con esa combinación habría menos errores y los cupos se controlarían mejor desde las primeras semanas.

Referencias

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., y Thomas, D. (2001). *Manifiesto por el desarrollo ágil de software*. https://agilemanifesto.org/iso/es/manifesto.html

Booch, G., Rumbaugh, J., y Jacobson, I. (2006). *El lenguaje unificado de modelado: Guía del usuario* (2.ª ed.; J. J. García Molina y J. Sáez Martínez, Trads.). Pearson Educación. (Obra original publicada en 2005)

Pressman, R. S. (2010). *Ingeniería del software: Un enfoque práctico* (7.ª ed.). McGraw-Hill.

Schwaber, K., y Sutherland, J. (2020). *La Guía Scrum*. https://scrumguides.org/scrum-guide.html

Sommerville, I. (2011). *Ingeniería de software* (9.ª ed.). Pearson Educación.